

Summary

In this milestone, I expanded the Eco Clean platform beyond appointment scheduling reliability and moved into core operational functionality. My focus was on implementing a calendar-driven workflow and building out the main business entities **users, clients, and staff** across the full stack. The goal was to make the system feel like a real admin tool: structured data, consistent UI patterns, and workflows that reflect how the business operates.

This milestone also included important architectural decisions, especially around reducing duplication across platforms and evaluating whether continuing with a standalone mobile app is the best long-term option.

1. Calendar Component Implementation

A major deliverable in this milestone was implementing the **calendar experience**, which acts as the central way to view and manage scheduled work.

What I did:

- I implemented a calendar component that supports viewing appointments in a structured, timeline-based way.
- I connected calendar interactions to the backend appointment data, so the calendar reflects the actual database state.
- I ensured time handling remains consistent with the earlier decision to store scheduling timestamps in UTC (TIMESTAMPTZ) so the calendar displays correct dates across timezones and daylight-saving changes.

The product shifted from “appointments exist in the database” to “appointments are visible and manageable in a real scheduling interface.”

2. Full-Stack Implementation of Users, Clients, and Staff

I built out **Users, Clients, and Staff** as first-class entities and connected them through the UI and backend so the system can support real business workflows.

Users (Admin / Staff Management)

- I implemented the full-stack flow to create and manage users.
- I ensured role-based structure exists (e.g., Admin vs Staff).
- I improved edit flows so updating user information is reliable and consistent.

Clients (Customer Records)

- I implemented full-stack client management and connected clients to scheduling workflows.
- I improved usability by enabling client lists that support scaling (searching/filtering and clean selection patterns).

Staff (Scheduling Assignments)

- I implemented staff listing and ensured staff can be assigned correctly to appointments.
- I strengthened the data model and flows, so staff assignments reflect real operations (not just UI placeholders).

The platform moved from a “scheduling prototype” into a structured business system where appointments connect to real **clients** and real **staff**, managed by real **users**.

3. UI Consistency and Reusability Improvements

To support the growing number of screens and flows, I continued improving UI consistency across the app.

I focused on:

- Creating reusable components and consistent layout patterns
- Standardizing form behavior and validation patterns

This reduced duplication and made new screens faster to build without sacrificing quality.

4. Deployment and Live Preview

During this milestone, we transitioned the Eco Clean platform from a development-focused environment to a production-ready deployment setup. Our goal was to ensure the system is stable, secure, and scalable in a real hosting environment rather than only functioning locally. Preview the live project through [this link](#).

This phase required architectural adjustments, environment configuration, and resolving several deployment-specific issues.

Key Decisions Made Along the Way

Decision 1: Keep Calendar as the Core Workflow View

- Instead of building isolated screens for “appointments,” I treated the calendar as the central workflow interface. This matches how scheduling businesses operate.

Decision 2: Build Entities Properly Before Expanding Features

- I prioritized building Users / Clients / Staff cleanly first, because these are foundational. Without them, adding advanced features would create messy workarounds later.

Decision 3: Contemplating Mobile App vs PWA

- As the web admin platform became more complete, I began questioning whether maintaining a separate mobile codebase is worth it long-term.

We started evaluating the possibility of replacing the mobile app with a Progressive Web App (PWA) because it could:

- Reduce duplicated development effort
- Simplify deployment and updates
- Keep one consistent UI and workflow across devices
- Speed up iteration and maintenance

We have not fully migrated yet, but this milestone is where that strategic thinking became a serious consideration.

Conclusion

This milestone was a turning point where I moved the platform closer to a production-ready operational tool. By implementing the calendar component and building Users, Clients, and Staff across the full stack, I established the core infrastructure needed for real scheduling operations.

The system is now more structured, scalable, and aligned with real business workflows while also setting up a future pathway where platform simplification via a PWA could reduce long-term complexity.

Work Logs

Date	Time	Description of Work Done
2026-02-08	6:00-8:00pm	Implemented client listing with Prisma pagination and sorting
2026-02-08	8:30-10:00pm	Integrated dynamic orderBy logic (oldest/newest toggle)
2026-02-09	4:30-6:30pm	Connected Mantine Select to backend client data
2026-02-09	7:00-9:00pm	Implemented searchable Select with onSearchChange filtering
2026-02-10	3:30-5:30pm	Structured dashboard layout using Mantine Grid and Stack
2026-02-10	6:00-8:30pm	Implemented sidebar collapse behavior (Mantine v6 responsive layout)
2026-02-11	2:00-4:00pm	Designed structured form layout (Primary Contact & section grouping)
2026-02-11	5:00-7:00pm	Connected form submission to backend API endpoints
2026-02-11	8:00-9:30pm	Improved spacing, hierarchy, and alignment for form UI
2026-02-12	1:00-3:00pm	Integrated FullCalendar with Mantine dashboard layout
2026-02-12	3:30-5:00pm	Configured calendar plugins (timeGrid, dayGrid, interaction)

2026-02-12	6:00-7:30pm	Debugged inconsistent page sizing and standardized container widths
2026-02-12	8:00-9:00pm	Generated database dump process for team collaboration
2026-02-12	9:00-10:00pm	Refactored reusable layout structure for scalability
2026-02-13	10:00am-12:00pm	Refactored Job–Appointment relationship logic (Prisma schema updates)
2026-02-13	2:00-3:30pm	Resolved Prisma migration constraint errors for required datetime fields
2026-02-13	7:00-9:00pm	Improved job creation transaction flow (Anytime vs Scheduled logic)
2026-02-16	3:00-5:00pm	Standardized date utilities (ISO ↔ date-only ↔ time conversions)
2026-02-17	11:00am-1:30pm	Reworked Anytime scheduling logic in Job POST route
2026-02-17	3:30-6:00pm	Fixed Prisma staff relation errors (staff vs staffId correction)
2026-02-18	9:00-11:30pm	Debugged appointment update errors (Invalid Date handling)
2026-02-18	11:30pm-12:30am	Normalized API date parsing to prevent runtime failures
2026-02-19	2:30-4:30pm	Fixed appointment PATCH route (async params resolution)
2026-02-19	5:00-6:30pm	Resolved DELETE route debugging for uploadthing integration
2026-02-19	7:00-9:00pm	Refactored line item pricing logic (unitCost vs quantity modeling)
2026-02-19	9:00-10:30pm	Streamlined API validation and frontend data normalization
2026-02-20	3:00-5:00pm	Improved appointment image upload handling (Mantine Dropzone integration)
2026-02-22	8:00-10:30pm	Planned PWA transition strategy (unified web/mobile architecture)
2026-02-23	9:30am-11:30am	Reviewed cloud scalability & CDN strategy for production readiness
2026-02-23	1:00-3:00pm	Evaluated infrastructure design for fault tolerance and automation
2026-02-25	6:30-7:15pm	Migrated database from local Postgres to Supabase production instance
2026-02-25	7:15-8:00pm	Reconfigured Prisma client for Vercel deployment
2026-02-25	8:00-8:30pm	Debugged Supabase connection error (P1001) and validated SSL/network configuration
2026-02-25	8:30-9:15pm	Fixed Prisma client generation issues on Vercel
2026-02-25	9:15-10:40pm	Refactored apiClient to fully generic typed implementation
2026-02-25	10:40-11:20pm	Resolved Prisma create() type conflicts

2026-02-25	11:20-11:50pm	Converted User modal into reusable strongly typed upsert component
2026-02-25	11:50-12:15am	Hardened optimistic updates and image deletion flow in AppointmentInfoModal
2026-02-26	10:00-12:00pm	Implemented calendar component and connected appointment data
2026-02-26	1:00-3:00pm	Integrated full-stack Users management
2026-02-26	4:00-6:00pm	Implemented Clients full-stack flow
2026-02-26	7:00-9:00pm	Structured Staff management and assignment logic
2026-02-27	9:30-11:30am	Refactored calendar rendering to align with UTC logic
2026-02-27	12:30-2:00pm	Resolved Prisma schema inconsistencies
2026-02-27	3:00-5:00pm	Stabilized navigation and layout for management screens
2026-02-27	6:30-8:30pm	Evaluated mobile app architecture and explored PWA strategy
Total	77h 15m	

AI Usage

AI	Prompt	Value Addition
ChatGPT 5.2 (Go Plan)	Prisma client.findMany with filtering, sorting, pagination	Built scalable backend query logic with dynamic orderBy and efficient skip/take pagination
ChatGPT 5.2 (Go Plan)	Implementing oldest/newest sorting toggle	Enabled dynamic server-side sorting with clean query structure
ChatGPT 5.2 (Go Plan)	Fetch data + total count simultaneously	Optimized performance using Promise.all() to prevent sequential DB calls
ChatGPT 5.2 (Go Plan)	Mantine Select with dynamic client data	Structured mapping of backend client records into Select component format
ChatGPT 5.2 (Go Plan)	Searchable Select with onSearchChange	Enabled live client filtering and improved UX for large datasets
ChatGPT 5.2 (Go Plan)	Handling page changes in pagination	Connected frontend pagination state with backend skip/take logic
ChatGPT 5.2 (Go Plan)	Sidebar collapse behavior (Mantine v6)	Implemented responsive sidebar with controlled layout resizing
ChatGPT 5.2 (Go Plan)	Maintaining consistent dashboard width across pages	Standardized Container sizing and layout structure
ChatGPT 5.2 (Go Plan)	Structuring complex form layout with Grid + Stack	Improved readability, spacing hierarchy, and maintainability

ChatGPT 5.2 (Go Plan)	Dividing form into logical sections	Increased UX clarity and better information grouping
ChatGPT 5.2 (Go Plan)	Connecting Mantine form to backend API	Clean separation between UI structure and API submission logic
ChatGPT 5.2 (Go Plan)	FullCalendar integration inside Mantine layout	Embedded scheduling system while preserving dashboard consistency
ChatGPT 5.2 (Go Plan)	FullCalendar plugins configuration	Enabled multi-view calendar functionality with proper plugin setup
ChatGPT 5.2 (Go Plan)	Matching UI to screenshot reference	Translated design references into Mantine component architecture
ChatGPT 5.2 (Go Plan)	Database dump for teammate	Provided environment sharing workflow for collaborative development
ChatGPT 5.2 Plus	Debug client edit form not populating	Corrected React Query data unwrapping and response shape mismatch
ChatGPT 5.2 Plus	Fix React Query POST vs PATCH issue	Eliminated stale closure issue preventing accidental duplication
ChatGPT 5.2 Plus	Design PATCH endpoint with nested addresses	Built transaction-safe Prisma update flow with relational consistency
ChatGPT 5.2 Plus	Debug 'Missing required fields' error	Identified double JSON stringification in apiClient
ChatGPT 5.2 Plus	Convert useEffect to React Query	Introduced scalable caching and structured query keys
ChatGPT 5.2 Plus	Implement query invalidation	Enabled automatic UI refresh after mutations
ChatGPT 5.2 Plus	Create typed API client layer	Standardized API interaction and removed unsafe any usage
ChatGPT 5.2 Plus	Define DTO types	Strengthened type safety and reduced runtime integration errors
ChatGPT 5.2 Plus	Fix Prisma client export errors on Vercel	Implemented postinstall prisma generate to prevent production build failures
ChatGPT 5.2 Plus	Configure Supabase with SSL for Prisma	Enabled secure production database connectivity
ChatGPT 5.2 Plus	Debug P1001 Supabase connection error	Diagnosed environment and connectivity configuration issues
ChatGPT 5.2 Plus	Refactor apiClient to generic typed pattern	Enabled fully type-safe API calls across the project
ChatGPT 5.2 Plus	Convert User modal to reusable upsert component	Reduced duplication and improved maintainability

ChatGPT 5.2 Plus	Ensure Node runtime for Prisma routes	Stabilized serverless execution in Vercel production
------------------	---------------------------------------	--

Repo Check In

Work on the repository was carried out on a regular basis, aligned with the project timeline and development milestones. Development activities included frontend UI components, backend API implementation, database schema evolution, and automation logic.

Majority of the work for the **eco-clean-web** and **eco-clean-mobile** was committed through the branches: **jobs**, **user_enhancements**, **web-api**, through multiple pull requests.

PR #5: https://github.com/vidarshan/W26_4495_S2_UpulA/pull/5

PR #7: https://github.com/vidarshan/W26_4495_S2_UpulA/pull/7

PR #8: https://github.com/vidarshan/W26_4495_S2_UpulA/pull/8

The work was checked into the repository almost every day, followed by a merge to main almost every 7 days to publish changes to the live preview

